DS 8001 — Lecture 6: Search Algorithms

Exercises with Worked Solutions

Exercise 1: Uninformed Search

Ex1: Uninformed Search

Consider a search tree in which every node has $b = 3$ successors and the maximum depth is $m = 6$. The shallowest goal is at depth $d = 3$ and is the last node generated at that depth (worst case); other goals may exist deeper in the tree. The goal test is applied when a node is generated. When counting generated nodes, do not count the root.

(a) How many nodes does BFS generate, up to and including the goal? BFS stores every generated node (with its back pointer) and the root: about how many nodes are in memory when the goal is found?

(b) Iterative deepening runs depth-limited DFS with limits 1, 2 and 3. How many nodes does it generate in total? Why is this only a constant factor more than BFS?

(c) DFS (without a depth limit) keeps only the nodes on its stack and discards a subtree once it has been fully explored. At most how many nodes are on its stack? Is DFS guaranteed to return the shallowest goal?

(d) Memory is limited to 20 nodes, and the shallowest goal is required. Which of BFS, DFS and iterative deepening would you use?

Solution

Setting up. The *input size* is described by the branching factor $b = 3$, the goal depth $d = 3$ and the maximum depth $m = 6$. We measure *time* by the number of generated nodes (the elementary operation is creating one successor) and *memory* by the number of nodes stored at the same time. Since every node has 3 children, depth k contains 3^k nodes: $1, 3, 9, 27, 81, 243, 729$ at depths $0, 1, \ldots, 6$.

(a) BFS uses a first-in first-out queue, so it generates the tree level by level. The goal test is applied at generation and the goal is the *last* node of depth 3, so BFS generates all of depths 1, 2 and 3 and nothing deeper (no depth-3 node is ever expanded):

$$\text{generated} \;=\; b + b^2 + b^3 \;=\; 3 + 9 + 27 \;=\; \boxed{39}.$$

In general BFS generates $b + b^2 + \cdots + b^d = O(b^d)$ nodes.

Memory. BFS keeps every generated node, because the back pointers are needed to read off the path once the goal is found. Together with the root this is $39 + 1 = \boxed{40}$ nodes. Of these, the 27 depth-3 nodes are still waiting in the queue; the other 13 have been expanded but are kept for their back pointers. BFS memory therefore grows like b^d, as fast as its running time.

(b) Iterative deepening (IDS) runs a depth-first search that never goes below a depth limit L, for $L = 1, 2, 3$. The run with limit L generates every node at depths $1, \ldots, L$; in the last run the goal is the last node of depth 3, so all of depths 1–3 is generated there too.

iteration	nodes generated	count
limit 1	depth 1	3
limit 2	depths 1, 2	3 + 9 = 12
limit 3	depths 1, 2, 3	3 + 9 + 27 = 39
total		$\boxed{54}$

Counting by depth instead: depth-1 nodes are generated in all 3 runs, depth-2 nodes in 2 runs and depth-3 nodes in 1 run, so the total is $3 \cdot 3 + 2 \cdot 9 + 1 \cdot 27 = 9 + 18 + 27 = 54$. This is the lecture's formula $d\,b + (d-1)\,b^2 + \cdots + 1 \cdot b^d$.

Why only a constant factor. IDS does $54/39 \approx 1.38$ times the work of BFS, i.e. 38% more. The repeated work is cheap because most nodes of a tree sit in its deepest level: depth 3 alone has 27 nodes, more than all shallower levels together ($1 + 3 + 9 = 13$). Re-generating the shallow levels repeats only a small part of the tree. In general $N_{\text{IDS}} \le b^d \big(b/(b-1)\big)^2$, which stays within a constant factor of BFS's $b^d\, b/(b-1)$; the factor gets smaller as b grows (about 11% extra for $b = 10$).

Each IDS run is a DFS of depth at most 3, so by the argument in (c) its stack holds at most $d(b-1)+1 = 2 + 2 + 3 = 7$ nodes.

(c) When DFS expands a node it pushes all 3 children onto the stack, continues with one of them, and leaves the other 2 waiting. Finished subtrees are discarded, so the stack holds only the waiting siblings along the current path. It is fullest just after a node at depth 5 (the deepest nodes that have children) is expanded: at that moment it holds the 2 waiting siblings at each of depths $1, \ldots, 5$ and the 3 new children at depth 6:

$$2 \cdot 5 + 3 \;=\; m(b-1) + 1 \;=\; \boxed{13}\ \text{nodes},$$

which is the lecture's $m(b-1) \sim O(bm)$ (counting the node being expanded as well). Memory is linear in the depth, not exponential.

Shallowest goal? $\boxed{\text{No.}}$ DFS returns the *first* goal it meets, and it always follows the current (leftmost) branch to its full depth before trying anything else. Suppose, for example, that there is another goal at depth 5 on the leftmost branch. DFS reaches it long before it gets to the depth-3 goal, which is the last node of depth 3 (in the rightmost part of the tree), and returns the deeper goal. On this finite tree DFS is complete (it will find *some* goal), but it is not optimal. In the worst case it generates $3 + 9 + \cdots + 729 = 1092$ nodes, i.e. $O(b^m)$ time.

(d) We need both ≤ 20 nodes in memory and the shallowest goal:

	memory (nodes)	fits in 20?	shallowest goal guaranteed?
BFS	40	no	yes
DFS	13	yes	no
iterative deepening	7	yes	yes

$\boxed{\text{Iterative deepening}}$ is the only choice that satisfies both requirements. It completes every depth L before moving to $L+1$, so the first goal it finds is a shallowest one, and it needs only DFS-like memory. The price is 54 instead of 39 generated nodes, a small constant factor.

Exercise 2: Informed Search

Ex2: Informed Search

Consider the search problem below, with start state S and goal state G. The entry in row X and column Y is the cost of the directed edge $X \to Y$ ("–" means there is no edge), and the last column gives the heuristic value $h(X)$.

from \ to	S	A	B	C	D	E	G	h
S	–	2	1	–	–	–	9	6
A	–	–	–	2	3	–	–	0
B	–	–	–	–	2	4	–	6
C	–	–	–	–	–	–	4	4
D	–	–	–	–	–	–	4	1
E	–	–	–	–	–	–	–	10
G	–	–	–	–	–	–	–	0

For each search below, give the order in which states are taken out of the priority queue (expanded), the path returned and its cost.

(a) Uniform-cost search (priority g)

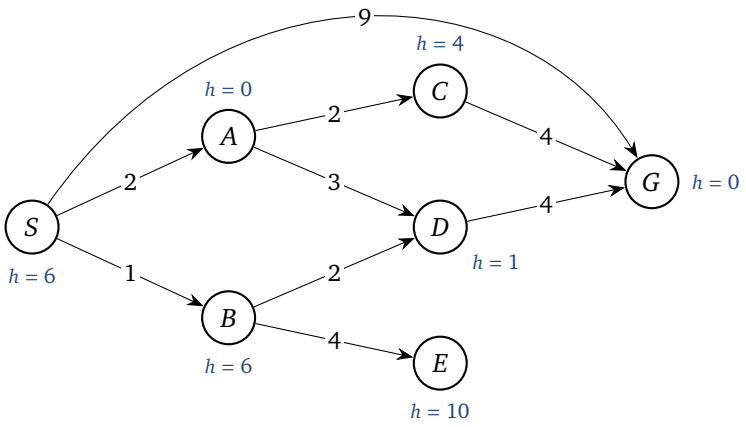
(b) Greedy best-first search (priority h)

(c) A search (priority $f = g + h$)

Conventions, as in the lecture's worked example: a state may enter the queue several times, once for each path that reaches it (tree search); the goal test is applied when a state is taken out of the queue; successors are added in alphabetical order, and ties are broken in favor of the entry added first.

Solution

Reading the table. Each non-empty entry is a directed edge, so the problem is the graph below; each state is labelled with its heuristic value h.



Notation. For a path we write g for its cost so far, h for the heuristic value of its last state, and $f = g + h$. A queue entry such as SBD (3) means "the path $S \to B \to D$ with priority 3". Each search

repeatedly takes out the entry with the smallest priority, stops if its last state is G, and otherwise adds one entry per successor. The three methods differ *only* in the priority.

(a) Uniform-cost search, priority g.

step	taken out	added	queue afterwards (in order)
1	S (0)	SA (2), SB (1), SG (9)	SB (1), SA (2), SG (9)
2	SB (1)	SBD (3), SBE (5)	SA (2), SBD (3), SBE (5), SG (9)
3	SA (2)	SAC (4), SAD (5)	SBD (3), SAC (4), SBE (5), SAD (5), SG (9)
4	SBD (3)	SBDG (7)	SAC (4), SBE (5), SAD (5), SBDG (7), SG (9)
5	SAC (4)	SACG (8)	SBE (5), SAD (5), SBDG (7), SACG (8), SG (9)
6	SBE (5)	none	SAD (5), SBDG (7), SACG (8), SG (9)
7	SAD (5)	SADG (9)	SBDG (7), SACG (8), SG (9), SADG (9)
8	SBDG (7)	goal test succeeds	

Points to notice:

- SBE (5) and SAD (5) tie in step 6; SBE entered the queue first (step 2), so it is taken out first.
- D is expanded twice, once through B ($g = 3$) and once through A ($g = 5$), because in tree search every path is a separate entry.
- A path to G (SG, cost 9) is in the queue from step 1, but it is accepted only when it is *taken out*. In step 8 the cheapest entry is $SBDG$ (7); every other entry costs at least 7, so no cheaper path can exist. Testing at generation would have wrongly returned $S \to G$ (cost 9).

Answer. Expansion order $\boxed{S, B, A, D, C, E, D}$, then G is taken out. Path $\boxed{S \to B \to D \to G}$ with cost $1+2+4 = \boxed{7}$, which is optimal; UCS always returns a cheapest path when the edge costs are positive.

(b) Greedy best-first search, priority h.

step	taken out	added	queue afterwards (in order)
1	S (6)	SA (0), SB (6), SG (0)	SA (0), SG (0), SB (6)
2	SA (0)	SAC (4), SAD (1)	SG (0), SAD (1), SAC (4), SB (6)
3	SG (0)	goal test succeeds	

SA and SG both have priority $h = 0$; SA entered first, so A is expanded first. With the opposite tie-break G would be taken out immediately after S, giving the same answer.

Answer. Expansion order $\boxed{S, A}$, then G is taken out. Path $\boxed{S \to G}$ with cost $\boxed{9}$, which is *not* optimal (cost 7 is possible). Greedy search ignores the cost already paid: the direct edge $S \to G$ costs 9, but $h(G) = 0$ makes G look perfect. The value $h(A) = 0$ is also misleading, since A is still 6 away from the goal.

(c) A search, priority $f = g + h$. The priorities are $f(SA) = 2 + 0$, $f(SB) = 1 + 6$, $f(SG) = 9 + 0$, $f(SAC) = 4+4$, $f(SAD) = 5+1$, $f(SADG) = 9+0$, $f(SBD) = 3+1$, $f(SBE) = 5+10$ and $f(SBDG) = 7+0$.

step	taken out	added	queue afterwards (in order)
1	S (6)	SA (2), SB (7), SG (9)	SA (2), SB (7), SG (9)
2	SA (2)	SAC (8), SAD (6)	SAD (6), SB (7), SAC (8), SG (9)
3	SAD (6)	SADG (9)	SB (7), SAC (8), SG (9), SADG (9)
4	SB (7)	SBD (4), SBE (15)	SBD (4), SAC (8), SG (9), SADG (9), SBE (15)
5	SBD (4)	SBDG (7)	SBDG (7), SAC (8), SG (9), SADG (9), SBE (15)
6	SBDG (7)	goal test succeeds	

Answer. Expansion order $\boxed{S, A, D, B, D}$, then G is taken out. Path $\boxed{S \to B \to D \to G}$ with cost $\boxed{7}$, which is optimal.

Why A finds the optimum here. Let $h^*(X)$ be the true cheapest cost from X to G. Working backwards from G: $h^*(C) = 4$, $h^*(D) = 4$, $h^*(A) = \min(2+4,\ 3+4) = 6$, $h^*(B) = \min(2+4,\ 4+\infty) = 6$ (no path from E reaches G, so $h^*(E) = \infty$) and $h^*(S) = \min(2+6,\ 1+6,\ 9) = 7$.

state	S	A	B	C	D	E	G
h	6	0	6	4	1	10	0
h^*	7	6	6	4	4	∞	0

Since $0 \le h \le h^*$ for every state, h is *admissible*, so this A search is in fact A*, which is guaranteed to return a cheapest path.

Why D is expanded twice. This h is not *consistent*: $h(S) = 6 > c(S,A) + h(A) = 2$ and $h(B) = 6 > c(B,D) + h(D) = 3$. As a result f can drop along a path (from 7 at SB to 4 at SBD), and D is first reached by the more expensive route (SAD, $f = 6$) and only later by the cheaper one (SBD, $f = 4$). Tree search simply expands both. If you had used a CLOSED list that never expands a state twice (lecture slide "If state space graph is not a tree"), the cheaper path to D would be thrown away and A would return $S \to A \to C \to G$ with cost 8. The lecture's A* algorithm avoids this by re-opening D (step 5.2), which again gives cost 7.

Summary.

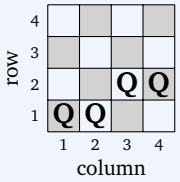
method	states expanded (then G taken out)	path	cost
uniform-cost	S, B, A, D, C, E, D	$S \to B \to D \to G$	7 (optimal)
greedy best-first	S, A	$S \to G$	9
A (= A* here)	S, A, D, B, D	$S \to B \to D \to G$	7 (optimal)

A finds the optimal path after 5 expansions, against 7 for uniform-cost search, because the heuristic steers it.

Exercise 3: Hill Climbing

Ex3: Hill Climbing

Consider 4-queens with one queen per column: $s = (r_1, r_2, r_3, r_4)$, where r_i is the row (1–4, bottom to top) of the queen in column i. Two queens attack each other if they share a row or a diagonal; $f(s)$ is the number of attacking pairs (minimize), and the number of conflicts of a queen is the number of queens it attacks. Use the neighborhood from the lecture: pick the most-conflicting column (the rightmost one, if tied) and move its queen to another row. Hill climbing moves to the best neighbor and stops when no neighbor is strictly better.



$s_0 = (1, 1, 2, 2)$

(a) Start from $s_0 = (1, 1, 2, 2)$. Compute $f(s_0)$ and the number of conflicts of each queen. Which column is chosen, and to which neighbor does hill climbing move?

(b) Continue until hill climbing stops. Check that $(2, 4, 1, 3)$ has $f = 0$. What kind of state has hill climbing stopped at?

(c) One run from a random start reaches $f = 0$ about 20% of the time. With independent random restarts, about how many runs (including the first) are needed on average? What is the probability that 10 runs all fail?

Solution

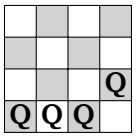
How to check a pair. Queens in columns $i < j$ attack each other if they are in the same row ($r_i = r_j$) or on the same diagonal ($|r_i - r_j| = j - i$: they are as many rows apart as columns apart). With 4 queens there are $\binom{4}{2} = 6$ pairs to check.

(a) For $s_0 = (1, 1, 2, 2)$:

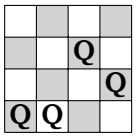
pair (columns)	rows	test	attack?
(1,2)	1, 1	same row	**yes**
(1,3)	1, 2	1 row apart, 2 columns apart	no
(1,4)	1, 2	1 row apart, 3 columns apart	no
(2,3)	1, 2	1 row apart, 1 column apart: diagonal	**yes**
(2,4)	1, 2	1 row apart, 2 columns apart	no
(3,4)	2, 2	same row	**yes**

So $f(s_0) =$ $\boxed{3}$. Each attacking pair adds one conflict to both of its queens, so the conflicts are queen 1: 1 (queen 2), queen 2: 2 (queens 1, 3), queen 3: 2 (queens 2, 4), queen 4: 1 (queen 3), i.e. $\boxed{(1, 2, 2, 1)}$.

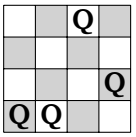
Columns 2 and 3 tie with 2 conflicts; by the tie rule we take the rightmost, $\boxed{\text{column 3}}$. Its queen (now in row 2) can move to rows 1, 3 or 4, giving three neighbors:



$(1, 1, \mathbf{1}, 2)$: $f = 4$
row 1: pairs (1,2),(1,3),(2,3); diagonal (3,4)



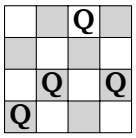
$(1, 1, \mathbf{3}, 2)$: $f = 3$
row: (1,2); diagonals: (1,3),(3,4)



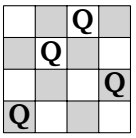
$(1, 1, \mathbf{4}, 2)$: $f = 1$
row: (1,2) only

The best neighbor is $(1, 1, 4, 2)$ with $f = 1 < 3$, so hill climbing moves to $\boxed{s_1 = (1, 1, 4, 2)}$.

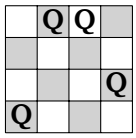
(b) In $s_1 = (1, 1, 4, 2)$ only queens 1 and 2 attack each other (same row), so $f(s_1) = 1$ and the conflicts are $(1, 1, 0, 0)$. Columns 1 and 2 tie, so the rightmost, $\boxed{\text{column 2}}$, is chosen; its queen moves from row 1 to row 2, 3 or 4:



$(1, \mathbf{2}, 4, 2)$: $f = 2$
diagonal (1,2); row (2,4)



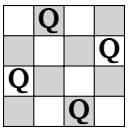
$(1, \mathbf{3}, 4, 2)$: $f = 1$
diagonal (2,3)



$(1, \mathbf{4}, 4, 2)$: $f = 2$
row (2,3); diagonal (2,4)

The best neighbor has $f = 1$, which is *not strictly better* than $f(s_1) = 1$, so hill climbing stops there: the final state is $\boxed{s_1 = (1, 1, 4, 2)}$ with $f = 1$.

Check of $(2, 4, 1, 3)$. The six pairs have rows $(2, 4)$, $(2, 1)$, $(2, 3)$, $(4, 1)$, $(4, 3)$, $(1, 3)$ for columns $(1, 2)$, $(1, 3)$, $(1, 4)$, $(2, 3)$, $(2, 4)$, $(3, 4)$. No two rows are equal, and the row differences $2, 1, 1, 3, 1, 2$ never equal the column differences $1, 2, 3, 1, 2, 1$. So $f(2, 4, 1, 3) = 0$: it is a solution.



Hill climbing therefore stopped at a $\boxed{\text{local minimum}}$ (a local optimum). No neighbor is better, yet the state is not a solution, because a state with $f = 0$ (the global minimum) exists. Hill climbing only sees the neighbors and cannot tell the difference. Note also that $(1, 3, 4, 2)$ has the same value $f = 1$; hill climbing as defined does not make such sideways moves.

(c) Each run succeeds with probability $p = 0.2$, independently of the other runs. Let K be the number of runs up to and including the first success. "$K = k$" means $k - 1$ failures followed by a success, so $P(K = k) = (1 - p)^{k-1} p$: K has a geometric distribution. Using $\sum_{k \geq 1} k\, x^{k-1} = 1/(1 - x)^2$ for $|x| < 1$,

$$E[K] = \sum_{k \geq 1} k\,(1 - p)^{k-1} p \;=\; p \cdot \frac{1}{\big(1 - (1 - p)\big)^2} \;=\; \frac{1}{p} \;=\; \frac{1}{0.2} \;=\; \boxed{5 \text{ runs}}.$$

All 10 runs fail with probability

$$(1 - p)^{10} = 0.8^{10} \approx \boxed{0.107},$$

so 10 random restarts find a solution with probability about 0.89. (The value 20% is realistic: running this hill climber from all $4^4 = 256$ possible start states succeeds about 21% of the time.)

Exercise 4: Simulated Annealing

Ex4: Simulated Annealing

Consider simulated annealing for a minimization problem. Let s be the current state, s' the neighbor picked as the candidate move, and $T(t)$ the temperature at time t. A move to a lower value, $f(s') < f(s)$, is always accepted; a move to a higher value is accepted with probability

$$p = \exp\Big(-\frac{|f(s') - f(s)|}{T(t)}\Big).$$

(a) If $f(s) = 6$, $f(s') = 5$ and $T(t) = 4$, and s' has been picked as the candidate, what is the probability that we move from s to s'? If instead the current state is s' and s has been picked, what is the probability that we move from s' to s?

(b) If $f(s) = 0$, $f(s') = \ln 5$ and $T(t) = 1$, and s' has been picked as the candidate, what is the probability that we move from s to s'?

Solution

The rule. We are *minimizing*, so lower values are better. Compare the candidate with the current state:

- candidate better (lower value): move with probability 1;
- candidate worse (higher value) by $\Delta = |f(\text{candidate}) - f(\text{current})| > 0$: move with probability $e^{-\Delta/T}$.

The probability is smaller when the move is much worse (large Δ) and when the temperature is low (small T).

(a) *From s to s':* $f(s') = 5 < f(s) = 6$, so the candidate is better, and the move is always accepted: probability $\boxed{1}$.

From s' to s: now the current state is s' with value 5 and the candidate s has value 6, which is worse by $\Delta = |6 - 5| = 1$. With $T = 4$,

$$p = e^{-1/4} = e^{-0.25} \approx \boxed{0.779}.$$

At this fairly high temperature a slightly worse move is accepted about 78% of the time. This is how simulated annealing escapes local optima.

(b) The candidate is worse: $\Delta = \ln 5 - 0 = \ln 5 \approx 1.609 > 0$. With $T = 1$,

$$p = e^{-\ln 5/1} = \frac{1}{e^{\ln 5}} = \frac{1}{5} = \boxed{0.2}.$$

For comparison, at the lower temperature $T = 0.5$ the same move would be accepted with probability $e^{-2\ln 5} = 1/25 = 0.04$. Lowering the temperature makes uphill moves rarer.

Exercise 5: Genetic Algorithm – 1

Ex5: Genetic Algorithm - 1

Each individual is a 5-bit string $x = (x_1, \ldots, x_5)$, and the reproduction probability of individual s_i is fitness-proportional: $p_i = f(s_i)/\sum_j f(s_j)$. Consider the population

A	B	C	D	E
$(0,0,1,0,0)$	$(0,1,0,0,1)$	$(0,0,1,1,0)$	$(0,0,0,1,0)$	$(0,0,0,0,0)$

(a) Let $f(x) = \min\{t \in \{1,2,3,4,5,6\} : x_t = 1\}$, where $x_6 = 1$ is an extra dummy bit (so $f(x)$ is the position of the first 1 in x, and $f(x) = 6$ if x contains no 1). Calculate the reproduction probabilities of $A, \ldots, E$.

(b) Let $f(x) = \max\{t \in \{0,1,2,3,4,5\} : x_t = 1\}$, where $x_0 = 1$ is an extra dummy bit (so $f(x)$ is the position of the last 1 in x, and $f(x) = 0$ if x contains no 1). Calculate the reproduction probabilities of $A, \ldots, E$.

Solution

Method. Fitness-proportional ("roulette-wheel") selection gives each individual a slice of a wheel whose size is proportional to its fitness: compute every $f(s_i)$, add them up, and divide each one by the total. The probabilities always add up to 1.

(a) Reading each string from left to right, f is the position of the first 1. String E has no 1 in positions 1–5, so the dummy bit $x_6 = 1$ gives $f(E) = 6$.

individual	string	first 1 at	f	$p_i = f/18$	decimal
A	$(0,0,1,0,0)$	position 3	3	$3/18 = 1/6$	0.167
B	$(0,1,0,0,1)$	position 2	2	$2/18 = 1/9$	0.111
C	$(0,0,1,1,0)$	position 3	3	$3/18 = 1/6$	0.167
D	$(0,0,0,1,0)$	position 4	4	$4/18 = 2/9$	0.222
E	$(0,0,0,0,0)$	none (x_6)	6	$6/18 = 1/3$	0.333
total			18	1	1.000

This fitness rewards strings that start with many zeros, so the all-zero string E is the most likely parent: $p_A = \boxed{1/6}$, $p_B = \boxed{1/9}$, $p_C = \boxed{1/6}$, $p_D = \boxed{2/9}$, $p_E = \boxed{1/3}$.

(b) Now f is the position of the *last* 1. String E has no 1 in positions 1–5, so the dummy bit $x_0 = 1$ gives $f(E) = 0$.

individual	string	last 1 at	f	$p_i = f/16$	decimal
A	$(0,0,1,0,0)$	position 3	3	3/16	0.1875
B	$(0,1,0,0,1)$	position 5	5	5/16	0.3125
C	$(0,0,1,1,0)$	position 4	4	$4/16 = 1/4$	0.25
D	$(0,0,0,1,0)$	position 4	4	$4/16 = 1/4$	0.25
E	$(0,0,0,0,0)$	none (x_0)	0	0	0
total			16	1	1

So $p_A =$ $\boxed{3/16}$, $p_B =$ $\boxed{5/16}$, $p_C =$ $\boxed{1/4}$, $p_D =$ $\boxed{1/4}$, $p_E =$ $\boxed{0}$. Individual E has fitness 0 and therefore probability 0: under proportional selection it can never become a parent. This is also why proportional selection needs $f \geq 0$; a negative fitness would give a negative "probability".

Comparing (a) and (b): the same population is ranked very differently (E is best in (a) and worst in (b)). The fitness function alone defines what a "good" individual is.

Exercise 6: Genetic Algorithm – 2

Ex6: Genetic Algorithm - 2

Consider the maximization problem

$$\max\ f(x) = -\frac{x^2}{10} + 3x, \qquad x \in \{0, 1, \ldots, 31\}.$$

Represent x by its 5-bit binary code, from $x = 0 \equiv (00000)$ to $x = 31 \equiv (11111)$. Starting from an initial population of 10 random solutions, illustrate two generations (steps) of a genetic algorithm that works on these bit strings:

- select 10 parents with fitness-proportional (roulette-wheel) selection, $p_i = f(s_i)/\sum_j f(s_j)$; since $f(31) = -3.1 < 0$, use fitness 0 for $x = 31$;
- pair the parents in order (1st with 2nd, 3rd with 4th, ...) and apply one-point crossover at a random cut point;
- flip each bit of each child independently with a small mutation probability (e.g. 0.02);
- the 10 children replace the old population.

For each generation, show the population, the fitness values, the selection probabilities, and the results of crossover and mutation.

Solution

Preparation. A bit string $b_1b_2b_3b_4b_5$ encodes $x = 16b_1 + 8b_2 + 4b_3 + 2b_4 + b_5$; for example 01101 is $8 + 4 + 1 = 13$. Writing $f(x) = x(30 - x)/10$ shows that f is 0 at $x = 0$ and $x = 30$, positive in between, and negative only at $x = 31$. Setting $f'(x) = 3 - x/5 = 0$ gives the maximum at $x = 15$, with $f(15) = 22.5$, encoded as 01111. Its neighbors $x = 14$ and $x = 16$ have $f = 22.4$.

About randomness. This exercise has no unique answer: every random choice (initial population, selection, cut points, mutations) can come out differently. Below is one complete run, with random numbers drawn by a computer. Any run in which the steps are carried out correctly is a valid answer. The random steps work as follows:

- *Selection:* draw u uniformly from $[0, 1)$ and pick the first individual whose cumulative probability exceeds u; repeat 10 times.
- *Crossover* with cut point $c \in \{1, 2, 3, 4\}$: child 1 takes the first c bits of parent 1 and the last $5 - c$ bits of parent 2; child 2 does the opposite.
- *Mutation:* each of the $10 \times 5 = 50$ bits flips with probability 0.02, so on average one bit flips per generation.

Generation 1

Population and selection probabilities (total fitness 158.3):

#	x	bits	$f(x)$	p_i	cumulative
1	13	01101	22.1	0.140	0.140
2	16	10000	22.4	0.142	0.281
3	24	11000	14.4	0.091	0.372
4	1	00001	2.9	0.018	0.390
5	12	01100	21.6	0.136	0.527
6	14	01110	22.4	0.142	0.668
7	25	11001	12.5	0.079	0.747
8	0	00000	0.0	0.000	0.747
9	16	10000	22.4	0.142	0.889
10	22	10110	17.6	0.111	1.000

For example, $p_1 = 22.1/158.3 = 0.140$. Individual 8 ($x = 0$) has fitness 0 and can never be selected; individual 4 ($x = 1$) is selected only rarely.

Selection. The ten random numbers and the parents they select:

u	0.71	0.96	0.03	0.67	0.85	0.37	0.11	0.35	0.79	0.10
parent #	7	10	1	7	9	3	1	3	9	1
x	25	22	13	25	16	24	13	24	16	13

For example, $u = 0.71$ lies between the cumulative values 0.668 (#6) and 0.747 (#7), so #7 is selected. Likewise $u = 0.67$ is just above 0.668 and also selects #7.

Crossover ("|" marks the cut) **and mutation:**

pair	parent 1	parent 2	cut	children
1	1100|1 (25)	1011|0 (22)	4	11000 (24), 10111 (23)
2	011|01 (13)	110|01 (25)	3	01101 (13), 11001 (25)
3	100|00 (16)	110|00 (24)	3	10000 (16), 11000 (24)
4	0|1101 (13)	1|1000 (24)	1	01000 (8), 11101 (29)
5	1000|0 (16)	0110|1 (13)	4	10001 (17), 01100 (12)

In pairs 2 and 3 both parents have the same tail after the cut, so the children are just copies of the parents: crossover creates something new only when the parents differ after the cut point. *Mutation:* one bit flipped, bit 1 of child 7: 01000 (8) $\rightarrow$ 11000 (24).

New population: 24, 23, 13, 25, 16, 24, 24, 29, 17, 12. The average fitness rose from 15.83 to 16.29; the best individual is still $x = 16$ ($f = 22.4$).

Generation 2

Population and selection probabilities (total fitness 162.9):

#	x	bits	$f(x)$	p_i	cumulative
1	24	11000	14.4	0.088	0.088
2	23	10111	16.1	0.099	0.187
3	13	01101	22.1	0.136	0.323
4	25	11001	12.5	0.077	0.400
5	16	10000	22.4	0.138	0.537
6	24	11000	14.4	0.088	0.626
7	24	11000	14.4	0.088	0.714
8	29	11101	2.9	0.018	0.732
9	17	10001	22.1	0.136	0.867
10	12	01100	21.6	0.133	1.000

Selection:

u	0.62	0.49	0.89	0.91	0.51	0.86	0.87	0.84	0.98	0.87
parent #	6	5	10	10	5	9	10	9	10	10
x	24	16	12	12	16	17	12	17	12	12

Crossover and mutation:

pair	parent 1	parent 2	cut	children
1	11|000 (24)	10|000 (16)	2	11000 (24), 10000 (16)
2	01|100 (12)	01|100 (12)	2	01100 (12), 01100 (12)
3	10|000 (16)	10|001 (17)	2	10001 (17), 10000 (16)
4	011|00 (12)	100|01 (17)	3	01101 (13), 10000 (16)
5	011|00 (12)	011|00 (12)	3	01100 (12), 01100 (12)

Mutation: one bit flipped, bit 3 of child 8: 10000 (16) → 10100 (20).

New population: 24, 16, 12, 12, 17, 16, 13, 20, 12, 12, with average fitness 20.98.

What the two generations show

Average fitness: 15.83 (initial) → 16.29 (after generation 1) → 20.98 (after generation 2); the best individual is $x = 16$ ($f = 22.4$) throughout.

- **Selection pushes the population upward.** Fit individuals are drawn more often, so the average fitness rose. With other random numbers it could also drop in some generation; improvement is likely, not guaranteed.
- **The optimum $x = 15$ (01111) has not been found yet.** The numbers 15 and 16 are neighbors, but their codes 01111 and 10000 differ in all five bits, so it is hard for bit operations to move from 16 to 15. From 13 = 01101, however, a single bit flip gives 01111. How the problem is encoded matters.
- **Diversity is shrinking.** After two generations $x = 12$ appears four times, and crossing a string with itself (pairs 2 and 5 of generation 2) produces nothing new. Mutation is what brings new bit values back; without enough diversity a GA can converge prematurely.